- *A new* `IJob` *instance is created every time the job is executed*. By contrast, the `BackgroundService` implementation is created only once, and its `Execute` method is invoked only once.
- *We can inject scoped dependencies directly into the* `IJob` *implementation*. To use scoped dependencies in the `IHostedService` implementation, we had to create our own scope manually and use service location to load dependencies. Quartz.NET takes care of that for us, allowing us to use pure constructor injection. Every time the job is executed, a new scope is created and used to create a new instance of the `IJob`.

The `IJob` defines what to execute, but it doesn't define when to execute it. For that, Quartz.NET uses triggers. *Triggers* can define arbitrarily complex blocks of time during which a job should execute. For example, you can specify start and end times, how many times to repeat, and blocks of time when a job should or shouldn't run (such as only 9 a.m. to 5 p.m. Monday to Friday).

In the following listing, we register the `UpdateExchangeRatesJob` with the DI container using the `AddJob<T>()` method, and we provide a unique name to identify the job. We also configure a trigger that fires immediately and then every five minutes until the application shuts down.

Listing 34.12 Configuring a Quartz.NET `IJob` and trigger

```
using Quartz;

IHost host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((hostContext, services) =>
    {
        services.AddQuartz(q =>
```